AI Concepts in DevOps · cheat sheet

DevOps Course · Session 6 · 1.3.27 · Noam Guy and Elhay Efrat

The agent in one picture

Agent = model + tools + loop + goal. **Model** decides, **context** is everything it sees on this call, **harness** runs the loop, executes tools and enforces rules.

1. Send the context to the model
2. The model replies with a tool call or a final answer
3. The harness checks permissions and runs the tool
4. The output is appended to the context; repeat

Stop conditions: goal done, max steps, budget or time limit, needs a human.

Context is a budget

- Every step re-sends the whole history: total input ≈ `N·S + (T+O)·N(N−1)/2`.
- Send slices, not dumps: `kubectl logs --tail=200 --since=1h`, `grep` before `cat`.
- Prompt caching bills the reused prefix at the cache-read price. Keep the prefix stable: no timestamps in the system prompt.
- Off track? Start a fresh session with a better first message.

Ask like you'd ask a senior colleague

What failed · where · the exact error · constraints · how to verify.

A minimal agent loop (Anthropic SDK)

```
def run_agent(goal: str, max_steps: int = 10) -> str:
    messages = [{"role": "user", "content": goal}]
    for _ in range(max_steps):
        response = client.messages.create(
            model=MODEL, max_tokens=16000, system=SYSTEM,
            tools=TOOLS, messages=messages,
        )
        messages.append({"role": "assistant", "content": response.content})
        if response.stop_reason != "tool_use":
            text = (b.text for b in response.content if b.type == "text")
            return next(text, f"Stopped: {response.stop_reason}")
        results = []
        for call in (b for b in response.content if b.type == "tool_use"):
            output = run_kubectl(call.input["args"])
            results.append({"type": "tool_result",
                            "tool_use_id": call.id, "content": output})
        messages.append({"role": "user", "content": results})
    return "Stopped: step limit reached without an answer"
```

Append the whole `response.content`, thinking blocks included. Run with `uv run --with anthropic agent.py`.

AGENTS.md skeleton

```
## Project
Python service, Docker, Helm, namespace: course.

## Commands
- Test: ./scripts/test.sh
- Deploy: helm upgrade --install course-app ./chart -n course

## Rules
- Run the tests before saying a task is done.
- Never run kubectl delete, helm uninstall, docker system prune.
- Images are non-root and pinned, never latest.
```

Guardrails in the harness

`.claude/settings.json` (Claude Code; other tools have the same idea)

```
{
  "permissions": {
    "allow": ["Bash(kubectl get:*)", "Bash(kubectl logs:*)"],
    "deny": ["Bash(kubectl delete:*)", "Read(./.env)"]
  },
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/guard.sh"
      }]
    }]
  }
}
```

```
#!/usr/bin/env bash
command=$(jq -r '.tool_input.command') || exit 2
blocked='--context[= ]prod|terraform.* destroy|kubectl.* delete( |$)'
if [[ "$command" =~ $blocked ]]; then
  echo "Blocked by policy: this needs a human" >&2
  exit 2
fi
```

Exit code 2 blocks the tool call and shows stderr to the model. `|| exit 2` fails closed if jq is missing. A regex is a speed bump; read-only credentials are the wall. Prompts are suggestions; hooks are rules.

Read-only Kubernetes identity

```
kubectl create serviceaccount ai-agent -n course
kubectl create rolebinding ai-agent-view -n course \
  --clusterrole=view --serviceaccount=course:ai-agent
TOKEN=$(kubectl create token ai-agent -n course --duration=1h)
kubectl config set-credentials ai-agent --token="$TOKEN"
kubectl config set-context ai-agent \
  --cluster="$CLUSTER" --user=ai-agent --namespace=course
kubectl --context ai-agent auth can-i delete pods
```

The built-in `view` role reads most resources, not Secrets. The last command answers `no`.

MCP on the wire (2026-07-28)

- JSON-RPC 2.0 over stdio (one JSON object per line) or Streamable HTTP (`MCP-Protocol-Version` header).
- Stateless: every request carries `_meta` with `io.modelcontextprotocol/protocolVersion` and `clientCapabilities`. Older servers (2025-11-25 and before) use an `initialize` handshake.
- `server/discover` → `tools/list` → `tools/call`.
- Tool failure: `"isError": true` in the result, so the model can retry. Bad request or unknown tool: JSON-RPC error `-32602`.

Graders, cheapest first

1. **Deterministic**: is-json, schema, icontains, regex, helm lint
2. **Semantic**: similarity to a reference answer
3. **LLM as judge**: a specific, pass/fail rubric
4. **Human**: label a sample to check the judge

Trust a judge only after measuring agreement with human labels: Cohen's $\kappa = (p_o - p_e) / (1 - p_e)$. Above 0.8 is almost perfect.

Structured outputs (`client.messages.parse(..., output_format=Model)`) guarantee the format; evals guard the content.

Evals as a CI gate (promptfoo)

```
prompts:
  - file://../prompts/explain.txt
providers:
  - id: anthropic:messages:claude-opus-5-5
defaultTest:
  options:
    provider: anthropic:messages:claude-opus-5-5
  assert:
    - type: is-json
      value: { type: object, required: [summary, severity] }
    - type: llm-rubric
      value: The fix is safe and never deletes data.
tests: file://cases.yaml
```

```
- name: Run evals
  env:
    ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
    PROMPTFOO_PASS_RATE_THRESHOLD: 90
  run: npx promptfoo eval -c evals/promptfooconfig.yaml -o report.html
```

Below the threshold promptfoo exits with code 100. Repeat flaky cases with `--repeat 3`. Current Claude models reject `temperature`: gate on a pass rate.

Seven controls for AI jobs in CI

1. Pin third-party actions to a commit SHA
2. Least-privilege `permissions:`, starting from `contents: read`
3. Secrets only on the step that needs them
4. Forks get no secrets: never run PR code with secrets via `pull_request_target`
5. Event data through `env:`, never pasted into `run:`
6. Bound the cost: `timeout-minutes`, `--max-turns`, a pass-rate gate
7. Treat AI output as untrusted: comment-only tokens, branch protection, a human merge

Six rules for AI at work

1. A human approves anything that writes
2. Read-only and least privilege by default
3. No secrets, customer data or prod credentials in prompts
4. Verify with tools: lint, test, scan, plan, dry-run
5. Review AI output like a junior's PR
6. Follow your company's AI policy

Lethal trifecta: private data + untrusted content + a way to send data out. Remove one leg.

OWASP Top 10 for LLM Applications 2025

`LLM01` Prompt Injection

`LLM02` Sensitive Information Disclosure

`LLM03` Supply Chain

`LLM04` Data and Model Poisoning

`LLM05` Improper Output Handling

`LLM06` Excessive Agency

`LLM07` System Prompt Leakage

`LLM08` Vector and Embedding Weaknesses

`LLM09` Misinformation

`LLM10` Unbounded Consumption

Try it at home

- **Free**: Ollama runs open models locally; promptfoo talks to it as `ollama:chat:qwen3`.
- **Paid, cheap**: one API key; a 10-case eval run costs well under a dollar.
- GitHub Models was retired in July 2026: if a tutorial uses it, swap the provider.

Read more

- `anthropic.com/engineering/building-effective-agents`
- `modelcontextprotocol.io · agents.md`
- `promptfoo.dev · github.com/confident-ai/deepeval`
- `genai.owasp.org · simonwillison.net (the lethal trifecta)`
- `platform.claude.com/docs · ollama.com · k8sgpt.ai`